KFKA – Entrega 2 de Web Full Stack: feito e pendente

Nov 4, 2026 · @André

Resumo

O código da Entrega 2 está pronto e testado; faltam **duas tarefas** para a entrega ficar completa: **subir o código no repositório** e **publicar o sistema na internet (deploy)**.

- **Prazo:** sexta, 06/11, até 23:59. Cada dia de atraso custa 25% da nota.
- **Onde está o código:** no arquivo `KFKA-E2-React.zip`, com o André. Ele tem as pastas `banco/`, `server/` (API) e `client/` (React), mais um `README.md` com as instruções.
- **Testes já feitos:** 74 testes da API e o cenário completo do PI no navegador (professor envia, admin publica, família vê e dá ciência), sem nenhuma falha.
- **Recomendação:** subir no repositório hoje ou amanhã cedo e publicar amanhã. Assim sexta fica livre para testar e corrigir.

Critérios da E2 x situação atual

Cinco dos sete critérios do documento do PI já estão cumpridos no código.

Critério da E2 (documento do PI)	Situação	Onde está / o que falta
API com rotas para as operações principais (listar, detalhar, criar etc.)	Pronto	Mais de 20 rotas em `server/rotas/` para os 3 perfis
Persistência de dados em banco	Pronto	MySQL, com transações em `server/modelos/FluxoAcompanhamento.js`
Autenticação (cadastro/login) e proteção de pelo menos uma rota	Pronto (login)	Login com JWT e todas as rotas protegidas por perfil. Ver "cadastro" em Pontos de atenção
Validação dos dados e tratamento de erros padronizado	Pronto	Todo erro sai como `{ "erro": "..." }`, com o status HTTP certo
Variáveis sensíveis fora do código	Pronto	`server/.env` (fora do Git) e `server/.env.example`

Critério da E2 (documento do PI)	Situação	Onde está / o que falta
Aplicação publicada (front e back juntos em produção)	**Falta**	Seção "Publicar o sistema"
Repositório com commits de todos os integrantes	**Falta**	Seção "Subir no repositório"

O que já foi feito

Na E2, o front virou um app React e a API foi completada e protegida. As tags, os `id`s e o CSS da E1 foram mantidos.

Estrutura do projeto

```
prototipoKFKA/
├── banco/     schema.sql (13 tabelas, índices, view) e seed.sql (dados fictícios)
├── server/    API: Node.js + Express + MySQL (mysql2) + JWT
└── client/    Front: React + Vite + React Router (padrão das aulas 06 a 08 e 12)
```

API (`server/`)

- Login com token JWT, que expira em 2h. As senhas são guardadas com bcrypt.
- Cada rota confere o token (401 sem login) e o perfil (403 com o perfil errado).
- O responsável só vê os alunos vinculados a ele. Isso corrigiu a falha da E1, em que a Mariana via o relatório do Igor.
- O fluxo de estados (rascunho → enviado → em revisão → publicado/devolvido/cancelado) fica na classe `FluxoAcompanhamento`. Toda mudança grava o histórico.
- Há exportação em CSV para o Excel (RF11) e a definição dos períodos dos bimestres (RF04).

Front (`client/`)

- Telas: login, 404, 4 do responsável, 4 do professor e 5 do admin (a 5ª é a nova tela "Períodos").
- O usuário logado fica no `AutenticacaoContexto` e todas as chamadas à API ficam em `servicos/`.
- Os formulários validam os campos antes de enviar. Publicar e cancelar pedem confirmação.

- O PDF do relatório sai pela impressão do navegador ("Salvar como PDF").

Banco (`banco/`)

- Novidades: a tabela `ciencia` (RF12) e a coluna `historico.observacao` (motivo da devolução).
- No seed, o 4º bimestre está aberto de 01/11 a 10/12/2026, para a entrega e a apresentação.

Falta 1: subir no repositório

O professor de Full Stack avalia pelo histórico do Git, então o código novo precisa entrar com commits dos dois responsáveis (André e Enzo). O histórico guarda a versão da E1, então nada se perde.

Antes de subir: rodar no computador

1. Recriar o banco com o schema e o seed novos (apaga só os dados fictícios antigos):
   - `mysql -u root -p -e "DROP DATABASE kfka;"`
   - `mysql -u root -p < banco/schema.sql`
   - `mysql -u root -p kfka < banco/seed.sql`
2. API: na pasta `server`, copiar `.env.example` para `.env`, preencher a senha do MySQL e um `JWT_SECRET`, rodar `npm install` e depois `npm run dev`. O endereço `http://localhost:3000/api/saude` deve responder `{"ok":true}`.
3. Front: em outro terminal, na pasta `client`, rodar `npm install` e depois `npm run dev`, e abrir `http://localhost:5173`. Entrar com `carla.lima@escola-teste.com` e a senha `senha123`.

Subir no repositório

1. Na raiz do repositório, apagar o que a E2 substituiu: a pasta `public/` e os arquivos `server.js`, `db.js`, `package.json`, `package-lock.json` e `.env.example` da raiz (eles agora estão dentro de `server/`). Também apagar o `schema.sql` e o `seed.sql` antigos, se estiverem fora de `banco/`.
2. Copiar do zip as pastas `banco/`, `server/` e `client/`, o `README.md` e o `.gitignore`.
3. Rodar `git status` e conferir que **não** aparecem `node_modules/` nem nenhum arquivo `.env`. Só o `.env.example` pode ir para o GitHub.
4. Dividir os commits por assunto, uma parte de cada um. Exemplos:
   - `banco: schema e seed da E2`
   - `server: API com JWT, rotas por perfil e classe FluxoAcompanhamento`
   - `client: front em React com React Router e Context`

- `docs: README de instalação e deploy`

5. Fazer o `git push` e conferir no site do GitHub se as três pastas apareceram.

- [ ] Banco recriado e site rodando no computador
- [ ] Código no repositório, sem `.env` e sem `node_modules`
- [ ] Commits do André e do Enzo no histórico

Falta 2: publicar o sistema (deploy)

São três serviços gratuitos, nesta ordem: banco no **Aiven**, API no **Render** e front no **Netlify**. Cada um precisa do endereço do anterior. Usem a versão mais nova do zip, que já aceita o certificado SSL do Aiven.

1. Banco MySQL no Aiven

O plano gratuito do Aiven inclui um MySQL e não pede cartão de crédito.

1. Criar uma conta em aiven.io e um serviço **MySQL**, plano **Free**.
2. Na tela do serviço (Overview), anotar: Host, Port, User e Password, e baixar o **CA certificate**.
3. Rodar o `banco/schema.sql` e depois o `banco/seed.sql` nesse banco. Pode ser pelo MySQL Workbench (nova conexão com SSL "Require") ou pelo terminal: `mysql -h <host> -P <porta> -u avnadmin -p --ssl-mode=REQUIRED < banco/schema.sql`.

2. API no Render

1. Em render.com, clicar em New → Web Service e escolher o repositório do GitHub.
2. Configurar:
   - Root Directory: `server`
   - Build Command: `npm install`
   - Start Command: `npm start`
   - Plano: Free
3. Em Environment, criar as variáveis do `.env.example`:
   - `DB_HOST`, `DB_PORT`, `DB_USER` e `DB_PASSWORD`, com os dados do Aiven
   - `DB_NAME=kfka`
   - `DB_SSL=true`
   - `DB_SSL_CA`: o texto inteiro do certificado, de `-----BEGIN CERTIFICATE-----` até o fim
   - `JWT_SECRET`: um texto longo e aleatório (o comando para gerar está no `.env.example`)
   - `JWT_EXPIRA=2h`

- `CORS_ORIGIN`: por enquanto `http://localhost:5173`

4. Esperar o deploy terminar e abrir `https://<seu-servico>.onrender.com/api/saude`. Tem que aparecer `{"ok":true}`.

3. Front no Netlify

1. Em netlify.com, clicar em Add new project → Import from Git e escolher o repositório.
2. Configurar:
   - Base directory: `client`
   - Build command: `npm run build`
   - Publish directory: `client/dist`
3. Em Environment variables, criar `VITE_API_URL` com o endereço do Render, **sem** `/` no fim. Exemplo: `https://kfka-api.onrender.com`.
4. Fazer o deploy. O arquivo `client/public/_redirects` já faz o F5 funcionar em qualquer tela.

4. Ligar os dois

1. Voltar ao Render e trocar `CORS_ORIGIN` pelo endereço do Netlify, sem `/` no fim (exemplo: `https://kfka.netlify.app`). Salvar: o Render reinicia sozinho.
2. Abrir o site do Netlify e fazer login. Se aparecer "Não foi possível conectar ao servidor", confiram `VITE_API_URL` e `CORS_ORIGIN`. São os dois erros mais comuns.

- [ ] Banco no Aiven com schema e seed
- [ ] API no Render respondendo `/api/saude`
- [ ] Front no Netlify fazendo login
- [ ] Endereços do site e da API anotados no `README.md` do repositório

Como testar o site publicado

O sistema está completo quando o cenário de ponta a ponta do PI (seção 10) funciona no site do Netlify. Todos os usuários de teste usam a senha `senha123`.

Perfil	E-mail	O que testar
Administrador	coordenacao@escola-teste.com	Revisar, alterar com justificativa, publicar, períodos, exportar CSV
Professor	carla.lima@escola-teste.com	Registrar, salvar rascunho, enviar, corrigir o devolvido (Bruno Lima)
Responsável	mariana.souza@exemplo.com	Mãe da Ana e do Pedro: relatórios, PDF, ciência

Perfil	E-mail	O que testar
Responsável	joao.pereira@exemplo.com	Pai do Igor. A Mariana **não** pode ver o Igor

Roteiro (também serve para a apresentação)

1. **Admin** → Períodos: conferir que o 4º bimestre está aberto.
2. **Professor** → Minhas turmas → 5º A – Matemática → "Registrar" na Ana Souza → preencher → Enviar para revisão.
3. **Admin** → Início → Revisar → mudar a média (pede justificativa) → Publicar. O histórico mostra cada passo.
4. **Responsável (Mariana)** → o relatório novo aparece em "Recentes" → abrir → Baixar em PDF → Registrar ciência.
5. **Professor** → abrir o mesmo acompanhamento e ver o retorno da família.
6. **Admin** → Revisão → Exportar (.csv) e abrir no Excel.

O teste muda dados do banco. Para deixar tudo como no início antes da apresentação, rodem de novo o `schema.sql` e o `seed.sql` no Aiven. Antes, apaguem o banco com `DROP DATABASE kfka;`.

Pontos de atenção

- **"Cadastro" no critério de autenticação:** o KFKA não tem auto-cadastro de propósito. Quem cria as contas é a escola (RF02), para que nenhum pai consiga se vincular sozinho a uma criança (LGPD). Vale confirmar com o professor se esse argumento basta. Se ele quiser ver um cadastro funcionando, dá para fazer uma tela de admin que cadastra usuários.
- **O Render gratuito "dorme"** depois de 15 minutos sem uso. O primeiro acesso depois disso pode levar de alguns segundos a mais de um minuto. Na apresentação, abram o site uns 2 minutos antes de começar.
- **Nunca subir o `.env`** nem colar senhas do Aiven no código. As credenciais ficam só nas variáveis de ambiente do Render.
- **Avaliação 360°** entre 07 e 08/11: ela multiplica a nota final, então os commits de cada um contam.
- **Fora desta entrega:** as telas de cadastro de alunos, responsáveis, professores e turmas (RF02/RF03) e a recuperação de senha por e-mail (RF01). Hoje esses dados vêm do `seed.sql`. Está anotado no `README.md`.

Dúvidas sobre o código: o `README.md` do zip explica as pastas, todas as rotas da API e a segurança.